

VNU-HCM University of Information Technology

ASSIGNMENT 4

By Chu Cuong - 24520236

Email: cuongchu200@gmail.com
6-21-2025

Mục lục

Lời nói đầu.....	2
1. Basic	3
1.1. Tree: Height of Tree	3
1.2. Binary Search Tree: Insert (Không dùng đệ quy)	4
1.3. Binary Search Tree: Insert.....	5
1.4. Tree: levelOrder Traversal – Duyệt cây BST theo chiều rộng.....	6
1.5. Tree: Inorder Traversal (LNR) II - Duyệt cây BST theo LNR không đệ quy.....	7
1.6. Tree: Inorder Traversal (LNR) - Duyệt cây BST theo LNR	8
1.7. Tree: Postorder Traversal (LRN) II - Duyệt cây BST theo LRN không đệ quy	9
1.8. Tree: Postorder Traversal (LRN) - Duyệt cây BST theo LRN.....	10
1.9. Binary Search Tree: Nút chung gần nhất	11
1.10. Tree: Top view.....	12
2. Advance	15
2.1. Ketban	15
2.2. HuffmanCoding.....	17
2.3. HoaNhac.....	20
2.4. Tree: levelOrder Traversal - Duyệt cây BST theo chiều rộng.....	22

GITHUB: [LINK](#)

Lời nói đầu

Bài tập tuần 4 có phần Basic với 10 bài tập được giao và phần Advance với 4 bài tập. Sau đây là phần trình bày và cách mình làm những bài này:

[IT003.P21.CTTN.2] Assignment 4 (Basic)

10 problems with a total score of 1000

#	PROBLEM	SCORE
1	Tree: Height of Tree	100
2	Binary Search Tree: Insert (không dùng đệ quy)	100
3	Binary Search Tree: Insert	100
4	Tree: levelOrder Traversal - Duyệt cây BST theo chiều rộng	100
5	Tree: Inorder Traversal (LNR) II - Duyệt cây BST theo LNR không đệ quy	100
6	Tree: Inorder Traversal (LNR) - Duyệt cây BST theo LNR	100
7	Tree: Postorder Traversal (LRN) II - Duyệt cây BST theo LRN không đệ quy	100
8	Tree: Postorder Traversal (LRN) - Duyệt cây BST theo LRN	100
9	Binary Search Tree: Nút chung gần nhất	100
10	Tree: Top view	100

[IT003.P21.CTTN.2] Assignment 4 (Advance)

4 problems with a total score of 400

#	PROBLEM	SCORE
1	khangtd.KetBan	100
2	khangtd.HuffmanCoding	100
3	khangtd.HoaNhac	100
4	Tree: levelOrder Traversal - Duyệt cây BST theo chiều rộng	100

Submit
upload source code

Choose File No file chosen

Select language
C++ (0.5s, 50MB)

Submit

[Code editor](#)

1. Basic

1.1. Tree: Height of Tree

➤ Đề bài:

Độ cao của cây nhị phân (Tree: Height of Binary Tree)

Sinh viên cài đặt hàm `int height(Node* root)` với `root` là con trỏ chỉ đến gốc của cây nhị phân ; hàm này sẽ tính độ cao của cây nhị phân.

Ví dụ:

Input	Output
1 \ 2 \ 5 / 3 6 \ 4	5

➤ Script:

```
int height(Node* root) {  
    if (root == NULL)  
        return 0;  
    int leftHeight = height(root->left);  
    int rightHeight = height(root->right);  
    return 1 + max(leftHeight, rightHeight);  
}
```

1.2. Binary Search Tree: Insert (Không dùng đệ quy)

➤ Đề bài:

Thêm vào cây nhị phân tìm kiếm (binary search tree)

Cài đặt hàm `Node * insert(Node * root, int data)` để thêm một node có giá trị là `data` vào cây nhị phân tìm kiếm có node gốc là tham số `root`, sau khi thêm vào thì cây đó vẫn là cây nhị phân tìm kiếm.

Chú ý : sinh viên không sử dụng đệ quy khi cài đặt hàm này

Ví dụ:

Hình ảnh cây nhị phân tìm kiếm trước và sau khi thêm node có `data` là 6.

Trước	Sau
<pre> 4 / \ 2 7 / \ 1 3</pre>	<pre> 4 / \ 2 7 / \ / 1 3 6</pre>

➤ Script:

```
Node * insert(Node * root, int data) {
    if(root == NULL)
        return new Node(data);
    else{
        Node *cur = root;
        while(true) {
            if(data <= cur->data) {
                if(cur->left == NULL) {
                    cur->left = new Node(data);
                    break;
                } else
                    cur = cur->left;
            } else {
                if(cur->right == NULL) {
                    cur->right = new Node(data);
                    break;
                } else
                    cur = cur->right;
            }
        }
    }
    return root;
}
```

1.3. Binary Search Tree: Insert

➤ Đề bài:

Thêm vào cây nhị phân tìm kiếm (binary search tree)

Cài đặt hàm `Node * insert(Node * root, int data)` để thêm một node có giá trị là `data` vào cây nhị phân tìm kiếm có node gốc là tham số `root`, sau khi thêm vào thì cây đó vẫn là cây nhị phân tìm kiếm.

Ví dụ:

Hình ảnh cây nhị phân tìm kiếm trước và sau khi thêm node có `data` là 6.

trước	sau
<pre> 4 / \ 2 7 / \ 1 3</pre>	<pre> 4 / \ 2 7 / \ / 1 3 6</pre>

➤ Script:

```
Node * insert(Node * root, int data) {
    if(root == NULL)
        return new Node(data);
    else {
        Node* cur;
        if(data <= root->data) {
            cur = insert(root->left, data);
            root->left = cur;
        } else {
            cur = insert(root->right, data);
            root->right = cur;
        }
        return root;
    }
}
```

1.4. Tree: levelOrder Traversal – Duyệt cây BST theo chiều rộng

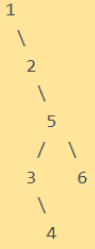
➤ Đề bài:

Tree: Level Order

Given a pointer to the root of a binary tree, you need to print the level order traversal of this tree. In level-order traversal, nodes are visited level by level from left to right. Complete the function and print the values in a single line separated by a space.

Sinh viên cài đặt hàm **void levelOrder(Node* root)** với root là con trỏ chỉ đến gốc của cây nhị phân ; hàm này sẽ in toàn bộ cây nhị phân thành một dòng các giá trị cách nhau với 1 khoảng trắng theo thứ tự duyệt theo chiều rộng.

Sample (Ví dụ):

Input	Output
	1 2 5 3 6 4

➤ Script:

```
void levelOrder(Node *root) {  
    if (root == NULL) return;  
  
    queue<Node*> q;  
    q.push(root);  
  
    while (!q.empty()) {  
        Node* cur = q.front(); q.pop();  
        cout << cur->data << " ";  
  
        if (cur->left != NULL)  
            q.push(cur->left);  
  
        if (cur->right != NULL)  
            q.push(cur->right);  
    }  
}
```

1.5. Tree: Inorder Traversal (LNR) II - Duyệt cây BST theo LNR không đệ quy

➤ Đề bài:

Tree: Inorder Traversal II (Duyệt cây theo thứ tự LNR)

Implement the **function void inOrder(Node* root)** where root is a pointer to the root of the binary tree; this function will print entire the binary tree in the LNR order traversal as a single line of the value of tree with a space separated. . Don't use recursion to solve this problem.

Sinh viên cài đặt hàm **void inOrder(Node* root)** với root là con trỏ chỉ đến gốc của cây nhị phân ; hàm này sẽ in toàn bộ cây nhị phân thành một dòng các giá trị cách nhau với 1 khoảng trắng theo thứ tự duyệt LNR. Sinh viên không được dùng đệ qui để giải bài này.

Sample (Ví dụ):

Input	Output
1 \ 2 \ 5 / 3 6 \ 4	1 2 3 4 5 6

➤ Script:

```
void inOrder(Node *root) {  
    stack<Node*> s;  
  
    while (root != NULL || !s.empty()) {  
        while (root != NULL) {  
            s.push(root);  
            root = root->left;  
        }  
  
        root = s.top(); s.pop();  
        cout << root->data << " ";  
        root = root->right;  
    }  
}
```


1.6. Tree: Inorder Traversal (LNR) - Duyệt cây BST theo LNR

➤ Đề bài:

Tree: Inorder Traversal (Duyệt cây theo thứ tự LNR)

Implement the **function void inOrder(Node* root)** where root is a pointer to the root of the binary tree; this function will print entire the binary tree in the LNR order traversal as a single line of the value of tree with a space separated.

Sinh viên cài đặt hàm **void inOrder(Node* root)** với root là con trỏ chỉ đến gốc của cây nhị phân ; hàm này sẽ in toàn bộ cây nhị phân thành một dòng các giá trị cách nhau với 1 khoảng trắng theo thứ tự duyệt LNR

Sample (Ví dụ):

Input	Output
1 \ 2 \ 5 / \ 3 6 \ 4	1 2 3 4 5 6

➤ Script:

```
void inOrder(Node *root) {  
    if(root == NULL) return;  
    inOrder(root->left);  
    cout << root->data << " ";  
    inOrder(root->right);  
}
```

1.7. Tree: Postorder Traversal (LRN) II - Duyệt cây BST theo LRN không đệ quy

➤ Đề bài:

Tree: Postorder Traversal (Duyệt cây theo thứ tự LRN)

Implement the **function void postOrder(Node* root)** where root is a pointer to the root of the binary tree; this function will print entire the binary tree in the LRN order traversal as a single line of the value of tree with a space separated. . Don't use recursion to solve this problem.

Sinh viên cài đặt hàm **void postOrder(Node* root)** với root là con trỏ chỉ đến gốc của cây nhị phân ; hàm này sẽ in toàn bộ cây nhị phân thành một dòng các giá trị cách nhau với 1 khoảng trắng theo thứ tự duyệt LRN. Sinh viên không dùng đệ qui khi giải bài này (dùng stack)

Sample (Ví dụ):

Input	Output
6 1 2 5 3 4 6	4 3 6 5 2 1

1 \ 2 \ 5 / \ 3 6 \ 4	

➤ Script:

```
void postOrder(Node *root) {  
    if (root == NULL)  
        return;  
  
    stack<Node*> s1, s2;  
    s1.push(root);  
  
    while (!s1.empty()) {  
        Node* cur = s1.top(); s1.pop();  
        s2.push(cur);  
        if (cur->left) s1.push(cur->left);  
        if (cur->right) s1.push(cur->right);  
    }  
  
    while (!s2.empty()) {  
        Node* cur = s2.top(); s2.pop();  
        cout << cur->data << " ";  
    }  
}
```

1.8. Tree: Postorder Traversal (LRN) - Duyệt cây BST theo

LRN

➤ Đề bài:

Tree: Postorder Traversal (Duyệt cây theo thứ tự LRN)

Implement the **function void postOrder(Node* root)** where root is a pointer to the root of the binary tree; this function will print entire the binary tree in the LRN order traversal as a single line of the value of tree with a space separated.

Sinh viên cài đặt hàm **void postOrder(Node* root)** với root là con trỏ chỉ đến gốc của cây nhị phân ; hàm này sẽ in toàn bộ cây nhị phân thành một dòng các giá trị cách nhau với 1 khoảng trắng theo thứ tự duyệt LRN.

Sample (Ví dụ):

Input	Output
6 1 2 5 3 4 6	4 3 6 5 2 1

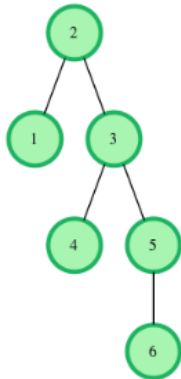
1 \ 2 \ 5 / 3 6 \ 4	

➤ Script:

```
void postOrder(Node *root) {  
    if(root == NULL) return;  
    postOrder(root->left);  
    postOrder(root->right);  
    cout << root->data << " ";  
}
```

1.9. Binary Search Tree: Nút chung gần nhất

➤ Đề bài:



Cho một cây nhị phân tìm kiếm (BST), sinh viên hãy cài đặt hàm *lca* để tìm ra nút tổ tiên thấp nhất của cây BST đã cho với các tham số.

- root: con trỏ trỏ đến nút gốc

- v1, v2: giá trị 2 nút cần tìm nút tổ tiên gần nhất

Hàm này sẽ trả về con trỏ trỏ đến nút tổ tiên gần nhất của 2 nút

Ví dụ

Input

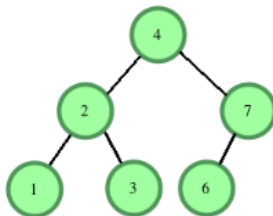
```
6
4 2 3 1 7 6
1 7
```

Output

Con trỏ trỏ đến nút có giá trị 4.

Diễn giải:

Sinh viên xem cây BST được tạo ra từ dữ liệu input ở trên



Với v1= 1 and v2= 7 thì hàm sẽ trả về con trỏ trỏ đến nút có giá trị là 4

➤ Ý tưởng:

Dễ thấy nếu cho hai nút có giá trị v1 và v2 thì nút chung của hai nút này là giá trị s sao cho: $\min(v1, v2) < s < \max(v1, v2)$ và nút đó phải nằm gần root nhất.

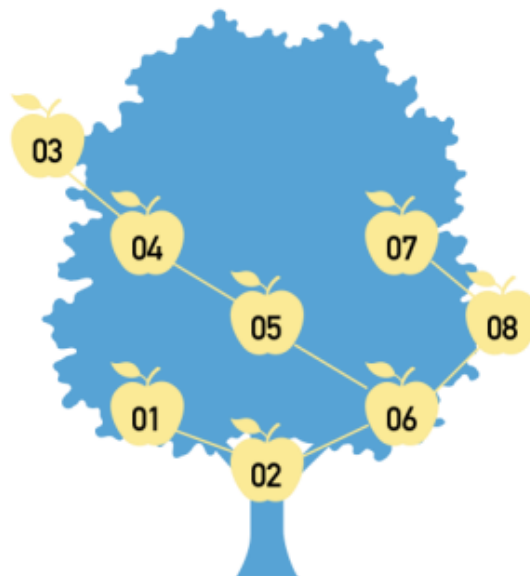
➤ Script:

```
Node *lca(Node *root, int v1,int v2) {
    if(root == nullptr || root->data == v1 || root->data == v2) return root;
    if(max(v1, v2) < root->data) return lca(root->left, v1, v2);
    else if(min(v1, v2) > root->data) return lca(root->right, v1, v2);
    return root;
}
```

1.10. Tree: Top view

➤ Đề bài:

Among the many juicy stone fruits in Vietnam, mangoes are one of the most commonly eaten fruits. Ti also has a mango tree in his backyard and this is an eccentric tree without leaves and totally flattened!!! There is only one root branch grown up from the trunk of the tree and from that branch has the most two other branches up, they are the same but one up to the left and one to the right. The branches continue to expand higher with one or two more grown up from it. On each branch we have a mango and each mango has its own value. The value of a mango on the right branch is always larger than the mango on its origin, and is smaller or equal for the mango on the left (see picture).



One morning, Ti came to his tree and tried to count the mangoes on the tree. From Ti standing spot - right to the bottom of the tree, the mangoes in the higher positions might be covered by the others in lower. Since the tree is tall and there are many mangoes on it, he could not follow all of them. Now, he wants you to help him, how many mangoes can he count from the bottom of the tree?

Input

The input consists of two lines. The first line is a positive integer n ($1 \leq n \leq 1.000.000$), - the number of mangoes. The second line is a sequence of n positive integers, each separates by a space - the value of mangoes. The value of mangos is listed by their positions. From the lower to higher and from the left to right.

Output

Print in one line the list of value of mangoes that can be observed in ascending order.

Sample input and output:

Input

8

2 1 6 5 8 4 7 3

Output:

1 2 3 6 8

➤ Ý tưởng:

- Duyệt cây theo chiều rộng (BFS)
- Gán cho mỗi node một tọa độ ngang (horizontal distance - pos):
 - Root có $pos = 0$.
 - Node trái có $pos + 1$.
 - Node phải có $pos - 1$.
- Với mỗi pos, chỉ ghi nhận node đầu tiên (trên cùng) được duyệt tại pos.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;
#define fi first
#define se second

class Node {
public:
    int data;
    Node *left;
    Node *right;
    Node(int d) {
        data = d;
        left = NULL;
        right = NULL;
    }
};

class Solution {
public:
    Node* insert(Node* root, int data) {
        if(root == NULL)
            return new Node(data);

        else {
            Node* cur;
            if(data <= root->data) {
                cur = insert(root->left, data);
                root->left = cur;
            } else {
                cur = insert(root->right, data);
                root->right = cur;
            }

            return root;
        }
    }
};

map<int, int> topview;

void view(Node* root) {
    if (root == NULL) return;
    queue<pair<Node*, int>> q;
    q.push({root, 0});
```

```

        while (!q.empty()) {
            Node* cur = q.front().fi;
            int pos = q.front().se; q.pop();

            if (topview.find(pos) == topview.end())
                topview[pos] = cur->data;

            if (cur->right) q.push({cur->right, pos - 1});
            if (cur->left) q.push({cur->left, pos + 1});
        }
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    Solution myTree;
    Node* root = NULL;

    int t, data;
    cin >> t;

    while(t-- > 0) {
        cin >> data;
        root = myTree.insert(root, data);
    }

    myTree.view(root);
    vector<int> res;
    for (auto& p : myTree.topview)
        res.push_back(p.se);

    sort(res.begin(), res.end());
    for (int i = 0; i < res.size(); i++)
        cout << res[i] << " ";

    return 0;
}

```

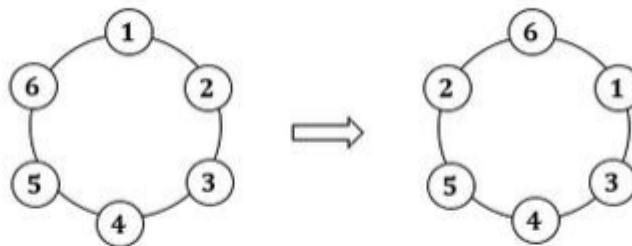
2. Advance

2.1. Ketban

➤ Đề bài:

Có một nhóm n sinh viên đang chơi trò chơi kết bạn. Các bạn sinh viên trên được đánh số từ 1 đến n và xếp thành hình vòng tròn từ 1 đến n theo chiều kim đồng hồ. Trong quá trình chơi, khi người quản trò hô "x kết bạn với y", thì bạn có số hiệu là x sẽ chạy đến đứng bên tay trái của bạn có số hiệu là y . Vòng tròn sau đó được điều chỉnh để tròn đều trở lại.

Bên dưới là vòng tròn ví dụ với $n = 6$ và câu hô là "2 kết bạn với 5".



Giả sử người quản trò hô câu kết bạn m lần. Hãy cho biết thứ tự của vòng tròn sau lần hô cuối cùng.

Dữ liệu:

- Dòng thứ nhất gồm hai số nguyên n, m cách nhau một khoảng trắng ($1 \leq n, m \leq 10^5$). n là số sinh viên và m là số lần hô kết bạn.
- m dòng tiếp theo, dòng thứ i gồm hai số nguyên x_i và y_i của lần hô thứ i ($1 \leq x_i, y_i \leq n$ và $x_i \neq y_i$).

Kết quả:

Là n số nguyên, mỗi số cách nhau một khoảng trắng, thể hiện số hiệu của các bạn sinh viên theo thứ tự trong vòng tròn sau m lần hô. Bạn được đếm đầu tiên là bạn có số hiệu 1.

Ví dụ:

INPUT		OUTPUT
6 1		1 3 4 5 2 6
2 5		

➤ Ý tưởng:

Tạo danh sách liên kết vòng và thay đổi vị trí trước và sau tương ứng với từng lần gọi để được vòng mới thỏa mãn yêu cầu đề bài.

➤ Script:

```
#include<bits/stdc++.h>
using namespace std;

int main(){
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int n, m;
    cin >> n >> m;
    vector<int> next(n + 1), prev(n + 1);

    for (int i = 1; i <= n; i++){
        next[i] = i % n + 1;
        prev[i % n + 1] = i;
    }

    for (int i = 0; i < m; i++){
        int tmp1, tmp2;
        cin >> tmp1 >> tmp2;

        // pos around tmp1
        prev[next[tmp1]] = prev[tmp1];
        next[prev[tmp1]] = next[tmp1];

        // pos between nexttmp2 and tmp1
        next[tmp1] = next[tmp2];
        prev[next[tmp2]] = tmp1;

        // pos between nexttmp2 and tmp1
        prev[tmp1] = tmp2;
        next[tmp2] = tmp1;
    }

    int cur = 1;
    do{
        cout << cur << " ";
        cur = next[cur];
    } while(cur != 1);

    return 0;
}
```

2.2. HuffmanCoding

➤ Đề bài:

Trong máy tính, khi lưu một văn bản xuống đĩa cứng, người ta thường dùng bản mã ASCII, một ký tự là 1 byte 8 bit. Ví dụ với câu văn bản sau: **A.PLAN.A.CANAL.PANAMA**

Câu văn bản trên có 21 ký tự. Do đó nếu biểu diễn bằng mã ASCII, thì chiều dài của câu văn trên là 21 byte (168 bit). Tuy nhiên với nhận xét rằng tần suất xuất hiện của các ký tự không đều, chữ A xuất hiện nhiều nhất (8 lần), sau đó là dấu . (4), chữ N (3), P, L (2), C, M (1), người ta có ý tưởng là dùng một từ mã ít bit nhất để biểu diễn ký tự A. Các chữ C, M ít xuất hiện hơn thì dùng từ mã dài hơn, như vậy tổng số bit để biểu diễn câu văn bản trên sẽ được rút ngắn lại. Xét các từ mã:

A biểu thị bởi **0**

Dấu chấm . biểu thị bởi **100**

N biểu thị bởi **110**

P biểu thị bởi **1010**

L biểu thị bởi **1011**

C biểu thị bởi **1110**

M biểu thị bởi **1111**

Các từ mã trên có tính chất là: một từ mã này **không là tiền tố** của bất cứ từ mã nào khác. Như vậy câu văn bản trên sẽ biểu diễn như sau:

0 100 1010 1011 0 110 100 0 100 1110 0 110 0 1011 100 1010 0 110 0 1111 0

(các khoảng trắng được thêm vào chỉ để dễ đọc, các bit khi lưu xuống đĩa cứng hay truyền qua mạng được lưu liên tục)

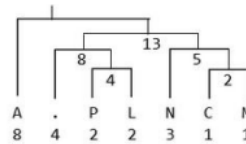
Do tính chất không tiền tố, từ dãy bit trên, ta có thể suy ra câu văn bản đầu (suy luận này là duy nhất, không có câu văn khác có cùng dãy bit trên). Số lượng bit của dãy trên là $8 \times 1 + 4 \times 3 + 3 \times 3 + (2+2+1+1) \times 4 = 53$ bit, ít hơn con số 168 bit của biểu diễn bằng mã ASCII. Đây là số bit ít nhất, không thể tìm bộ từ mã nào khác cho ra tổng số bit biểu diễn ít hơn 53 được. Phương pháp rút gọn biểu diễn như trên được gọi là phương pháp nén Huffman. Quá trình ngược lại từ dãy bit suy ra lại câu văn bản ban đầu gọi là giải nén. Phương pháp nén Huffman được sử dụng trong các chuẩn nén dữ liệu phổ biến như ảnh JPEG, ảnh PNG, file ZIP...

Người ta sử dụng cây nhị phân để tìm từ mã Huffman. Có thể minh họa quá trình tạo cây nhị phân xây dựng bộ mã Huffman cho câu văn trên như sau:

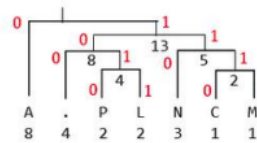
1) Thiết lập N nút lá của cây, mỗi nút là một chữ cái cùng với tần suất xuất hiện:

A	.	N	P	L	C	M
8	4	3	2	2	1	1

2) Dựng cây Huffman từ dãy tần suất trên:



3) Đánh số tất cả các nút con bên trái là **0**, tất cả các nút con bên phải là **1**:



4) Xuất phát từ nút gốc, lần lượt duyệt đến các nút lá, các số **0, 1** trên đường đi từ nút gốc đến nút lá chính là từ mã Huffman cho các nút lá tương ứng.

Trong quá trình tạo cây, tùy theo cách chọn 2 nút con mà ta có thể có các bộ mã Huffman khác nhau. Tuy nhiên các bộ mã này cùng cho ra biểu diễn nên có số bit đều là ít nhất.

Yêu cầu: Cho một câu văn bản, hãy nén câu văn bản trên dùng phương pháp nén Huffman và cho biết số bit có được sau khi nén.

Dữ liệu : gồm 2 dòng

- Dòng thứ nhất là số nguyên **N**, chiều dài của câu văn bản ban đầu ($1 \leq N \leq 10.000$)

- Dòng thứ hai là câu văn bản cần nén có chiều dài **N**, chỉ bao gồm các chữ cái là tính, dấu chấm '.' và dấu phẩy ',', ngoài ra không có các ký tự khác.

Kết quả:

- Là một số nguyên thể hiện số bit sau khi nén câu văn bản.

Ví dụ:

INPUT	OUTPUT
21 A.PLAN.A.CANAL.PANAMA	53

➤ Ý tưởng:

- Dùng `unordered_map<char, int>` để đếm số lần xuất hiện của từng ký tự.
- Tạo cây Huffman bằng `priority_queue` để lấy ra 2 node có tần số nhỏ nhất.
- Duyệt cây và sinh mã nhị phân theo từng ký tự bằng hàm `build()`.
- Tổng chiều dài của chuỗi sau khi mã hóa là kết quả cần tìm.

➤ Script:

```
#include<bits/stdc++.h>
using namespace std;
#define fi first
#define se second

struct Node {
    char ch;
    int freq;

    Node* left;
    Node* right;

    Node(char c, int f) {
        ch = c;
        freq = f;
        left = right = NULL;
    }
    Node(int f, Node* l, Node* r) {
        ch = 0;
        freq = f;
        left = l, right = r;
    }
};

struct comp {
    bool operator()(Node* a, Node* b) {
        return a->freq > b->freq;
    }
};

void build(Node* node, string bin, unordered_map<char, string>& binMp) {
    if (!node) return;

    if (node->left == NULL && node->right == NULL) {
        if (bin == "") bin = "0";
        binMp[node->ch] = bin;
        return;
    }

    build(node->left, bin + "0", binMp);
    build(node->right, bin + "1", binMp);
}
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    int n;
    string s;
    cin >> n >> s;

    unordered_map<char, int> freqMap;
    for (char c : s) freqMap[c]++;

    priority_queue<Node*, vector<Node*>, comp> pq;
    for (auto it : freqMap) pq.push(new Node(it.fi, it.se));

    if (pq.size() == 1) {
        Node* uniq = pq.top();
        cout << uniq->freq << endl;
        return 0;
    }
    while (pq.size() > 1) {
        Node* left = pq.top(); pq.pop();
        Node* right = pq.top(); pq.pop();
        Node* parent = new Node(left->freq + right->freq, left, right);
        pq.push(parent);
    }

    Node* root = pq.top();
    unordered_map<char, string> binMp;
    build(root, "", binMp);

    int res = 0;
    for (char c : s) res += binMp[c].length();
    cout << res << endl;

    return 0;
}

```

2.3. HoaNhac

➤ Đề bài:

Có N người đang chờ đợi xếp hàng để vào một buổi hòa nhạc. Mọi người cảm thấy buồn chán khi chờ đợi nên họ tìm những người thân của họ trong hàng đang xếp. Hai người **A** và **B** có thể nhìn thấy nhau nếu họ đang đứng ngay cạnh nhau hoặc không có người nào cao hơn **A** hoặc **B** đứng giữa. Hãy xác định số cặp có thể nhìn thấy nhau?



Dữ liệu:

- Dòng thứ nhất gồm số nguyên N ($1 \leq N \leq 10^5$). N là số người đang xếp hàng.
- Dòng tiếp theo gồm N số nguyên $h_1, h_2, \dots, h_N$ mỗi số cách nhau một khoảng trắng ($1 \leq h_i < 2^{31}$). h_i là chiều cao của người thứ i trong hàng.

Kết quả:

Là số lượng cặp người có thể thấy nhau ở trong hàng.

Ví dụ:

INPUT		OUTPUT
7 2 4 1 2 2 5 1		10

➤ Ý tưởng:

Duyệt từng người với chiều cao cur từ trái sang phải:

- Duy trì stack <pair<int, int>> lưu cặp người đứng (chiều cao và số lần liên tiếp).
- Khi gặp người mới:
 - Nếu người này cao hơn người đứng trước thì cộng kết quả vào rồi bỏ ra.
 - Nếu bằng nhau cộng số lần vào kết quả, rồi gộp số lượng ($cnt = freq + 1$).
 - Nếu trong stack vẫn còn người cao hơn thì cộng 1 vào kết quả.
- Đẩy cặp người hiện tại và cnt vào stack.

➤ Script:

```
#include<bits/stdc++.h>
using namespace std;
#define fi first
#define se second

int main(){
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    int n;
    cin >> n;
    vector<int> v(n);

    for(int i = 0; i < n; i++)
        cin >> v[i];

    stack<pair<int, int>> st;
    long long res = 0;

    for (int cur : v) {
        int cnt = 1;

        while (!st.empty() && st.top().fi < cur) {
            res += st.top().se;
            st.pop();
        }

        if (!st.empty() && st.top().fi == cur) {
            int freq = st.top().se;
            res += freq;
            cnt = freq + 1;
            st.pop();
        }

        if (!st.empty()) res++;

        st.emplace(cur, cnt);
    }

    cout << res;
    return 0;
}
```

2.4. Tree: levelOrder Traversal - Duyệt cây BST theo chiều rộng

→ lặp bài 1.4. phần Basic...